

C++

Programación en C++ de Cero a Matrices

36 ejercicios resueltos y explicados,
del primer "Hola mundo" hasta las matrices.

36

EJERCICIOS

6

NIVELES

100%

CON SOLUCIÓN

Bienvenido a este libro de práctica

Este pequeño libro nació para acompañar el curso de C++ del canal **Miguel Borja Academy**. Aquí no encontrarás teoría interminable, sino lo que de verdad te convierte en programador: **practicar resolviendo problemas**.

Los 36 ejercicios están ordenados por dificultad, desde lo más básico hasta el manejo de matrices. Cada uno incluye el enunciado, una solución completa lista para compilar y una breve explicación de la idea clave. Avanza a tu ritmo y no tengas miedo de equivocarte: cada error es un paso más cerca de dominar el lenguaje.

Cómo aprovechar este libro al máximo:

- Lee el problema e **intenta resolverlo tú mismo antes** de mirar la solución.
- Escribe el código a mano o en tu compilador; no copies y pegues.
- Cuando funcione, compáralo con la solución y lee la explicación.
- Modifica cada ejercicio: cambia valores, agrega validaciones, hazlo tuyo.

Puedes compilar cualquier ejercicio con un compilador como g++: guarda el código en un archivo programa.cpp y ejecuta `g++ programa.cpp -o programa`, luego `./programa`. ¡Manos a la obra!

Acompáñame en este viaje 🐼

- ▶ YouTube: youtube.com/@MiguelBorjaAcademy
- ★ GitHub: github.com/migueldevbp
- 📁 Repositorio de este libro: github.com/migueldevbp/c-workbook

Contenido

Nivel 1 — Fundamentos

- 01 Hola, mundo
- 02 Suma de dos números
- 03 Área de un rectángulo
- 04 Convertir Celsius a Fahrenheit
- 05 Promedio de tres notas
- 06 Intercambiar dos variables

Nivel 2 — Decisiones (condicionales)

- 07 Par o impar
- 08 El mayor de tres números
- 09 Positivo, negativo o cero
- 10 Calculadora simple con switch
- 11 Año bisiesto
- 12 Clasificar una nota

Nivel 3 — Repeticiones (bucles)

- 13 Tabla de multiplicar
- 14 Suma de 1 hasta N
- 15 Factorial de un número
- 16 Serie de Fibonacci
- 17 ¿Es número primo?
- 18 Invertir un número
- 19 Contar los dígitos

Nivel 4 — Funciones

- 20 Función potencia
- 21 Máximo común divisor (MCD)
- 22 Verificar número palíndromo
- 23 Factorial recursivo

Nivel 5 — Arreglos (vectores)

- 24 Llenar e imprimir un arreglo
- 25 Mayor y menor de un arreglo
- 26 Suma y promedio de un arreglo
- 27 Búsqueda lineal
- 28 Ordenar con método burbuja
- 29 Invertir un arreglo

Nivel 6 — Matrices (avanzado)

- 30 Llenar e imprimir una matriz
- 31 Suma de dos matrices
- 32 Multiplicación de matrices
- 33 Transpuesta de una matriz
- 34 Suma de la diagonal principal
- 35 Mayor elemento de la matriz
- 36 ¿Es matriz identidad?

SECCIÓN 1

Nivel 1 — Fundamentos

Tus primeros programas: entrada, salida, variables y operadores.

01 Hola, mundo

PROBLEMA

Escribe un programa que muestre en pantalla el mensaje **¡Hola, mundo!**. Es el primer paso de todo programador.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    cout << "¡Hola, mundo!" << endl;
    return 0;
}
```

💡 `cout` envía texto a la consola y `endl` hace un salto de línea. Todo programa en C++ inicia en `main()`.

02 Suma de dos números

PROBLEMA

Pide al usuario dos números enteros y muestra su suma.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Ingrese el primer numero: ";
    cin >> a;
    cout << "Ingrese el segundo numero: ";
    cin >> b;
    cout << "La suma es: " << a + b << endl;
    return 0;
}
```

💡 `cin` lee datos del teclado y los guarda en las variables. Aquí sumamos directamente al imprimir.

03 Área de un rectángulo

PROBLEMA

Lee la base y la altura de un rectángulo (números decimales) y calcula su área.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    double base, altura;
    cout << "Ingrese la base: ";
    cin >> base;
    cout << "Ingrese la altura: ";
    cin >> altura;
    double area = base * altura;
    cout << "El area es: " << area << endl;
    return 0;
}
```

💡 Usamos `double` para permitir decimales. El área de un rectángulo es $\text{base} \times \text{altura}$.

04 Convertir Celsius a Fahrenheit

PROBLEMA

Lee una temperatura en grados Celsius y conviértela a Fahrenheit usando la fórmula $F = C \times 9/5 + 32$.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    double celsius;
    cout << "Temperatura en Celsius: ";
    cin >> celsius;
    double fahrenheit = celsius * 9.0 / 5.0 + 32;
    cout << celsius << " C equivalen a " << fahrenheit << " F" << endl;
    return 0;
}
```

💡 Escribimos `9.0` y `5.0` (con decimal) para forzar una división real y no una división entera.

05 Promedio de tres notas

PROBLEMA

Lee tres notas y muestra su promedio.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    double n1, n2, n3;
    cout << "Ingrese tres notas: ";
    cin >> n1 >> n2 >> n3;
    double promedio = (n1 + n2 + n3) / 3.0;
    cout << "El promedio es: " << promedio << endl;
    return 0;
}
```

💡 Podemos leer varias variables seguidas con un solo `cin`. Los paréntesis aseguran que primero se suma y luego se divide.

06 Intercambiar dos variables

PROBLEMA

Lee dos números y muestra sus valores intercambiados usando una variable auxiliar.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int a, b, aux;
    cout << "Ingrese dos numeros: ";
    cin >> a >> b;

    aux = a;    // guardamos a
    a = b;      // a toma el valor de b
    b = aux;    // b toma el valor original de a

    cout << "a = " << a << ", b = " << b << endl;
    return 0;
}
```

💡 La variable `aux` evita perder el valor original de `a` al sobrescribirlo.

Nivel 2 — Decisiones (condicionales)

Aprende a que el programa tome decisiones con if, else y switch.

07 Par o impar

PROBLEMA

Lee un número entero e indica si es par o impar.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Ingrese un numero: ";
    cin >> n;
    if (n % 2 == 0)
        cout << n << " es par" << endl;
    else
        cout << n << " es impar" << endl;
    return 0;
}
```

💡 El operador % da el residuo de la división. Si el residuo entre 2 es 0, el número es par.

08 El mayor de tres números

PROBLEMA

Lee tres números y muestra cuál es el mayor.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Ingrese tres numeros: ";
    cin >> a >> b >> c;

    int mayor = a;
    if (b > mayor) mayor = b;
    if (c > mayor) mayor = c;

    cout << "El mayor es: " << mayor << endl;
    return 0;
}
```

💡 Suponemos que el primero es el mayor y lo comparamos con los demás, actualizándolo cuando encontramos uno más grande. Esta técnica sirve para cualquier cantidad de números.

09 Positivo, negativo o cero

PROBLEMA

Lee un número y clasifícalo como positivo, negativo o cero.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Ingrese un numero: ";
    cin >> n;
    if (n > 0)
        cout << "Positivo" << endl;
    else if (n < 0)
        cout << "Negativo" << endl;
    else
        cout << "Es cero" << endl;
    return 0;
}
```

💡 El `else if` permite encadenar varias condiciones; solo se ejecuta el primer bloque cuya condición sea verdadera.

10 Calculadora simple con switch

PROBLEMA

Lee dos números y un operador (+, -, *, /) y muestra el resultado de la operación.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    char op;
    cout << "Ingrese: numero operador numero (ej: 5 + 3): ";
    cin >> a >> op >> b;

    switch (op) {
        case '+': cout << "=" << a + b << endl; break;
        case '-': cout << "=" << a - b << endl; break;
        case '*': cout << "=" << a * b << endl; break;
        case '/':
            if (b != 0) cout << "=" << a / b << endl;
            else cout << "Error: division entre cero" << endl;
            break;
        default: cout << "Operador no valido" << endl;
    }
    return 0;
}
```

💡 `switch` compara una variable contra varios `case`. El `break` evita que se ejecuten los casos siguientes. Siempre validamos la división entre cero.

11 Año bisiesto

PROBLEMA

Lee un año e indica si es bisiesto. Un año es bisiesto si es divisible entre 4, salvo los múltiplos de 100 que no sean múltiplos de 400.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int anio;
    cout << "Ingrese un anio: ";
    cin >> anio;

    if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0))
        cout << anio << " es bisiesto" << endl;
    else
        cout << anio << " NO es bisiesto" << endl;
    return 0;
}
```

💡 Combinamos condiciones con `&&` (y) y `||` (o). Los paréntesis dejan clara la prioridad de la regla.

12 Clasificar una nota

PROBLEMA

Lee una nota del 0 al 20 y muestra su categoría: 0-10 Desaprobado, 11-14 Regular, 15-17 Bueno, 18-20 Excelente.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int nota;
    cout << "Ingrese la nota (0-20): ";
    cin >> nota;

    if (nota < 0 || nota > 20)
        cout << "Nota invalida" << endl;
    else if (nota <= 10)
        cout << "Desaprobado" << endl;
    else if (nota <= 14)
        cout << "Regular" << endl;
    else if (nota <= 17)
        cout << "Bueno" << endl;
    else
        cout << "Excelente" << endl;
    return 0;
}
```

💡 Primero validamos el rango. Luego, gracias al orden de los `else if`, basta comparar con el límite superior de cada categoría.

SECCIÓN 3

Nivel 3 — Repeticiones (bucles)

Repite tareas con `for` y `while` para resolver problemas más potentes.

13 Tabla de multiplicar

PROBLEMA

Lee un número y muestra su tabla de multiplicar del 1 al 12.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Ingresa un numero: ";
    cin >> n;
    for (int i = 1; i <= 12; i++) {
        cout << n << " x " << i << " = " << n * i << endl;
    }
    return 0;
}
```

💡 El bucle `for` tiene tres partes: inicio (`i = 1`), condición (`i <= 12`) y avance (`i++`). Se repite mientras la condición sea verdadera.

14 Suma de 1 hasta N

PROBLEMA

Lee un número N y calcula la suma $1 + 2 + 3 + \dots + N$.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n, suma = 0;
    cout << "Ingresa N: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        suma += i; // equivale a: suma = suma + i
    }
    cout << "La suma es: " << suma << endl;
    return 0;
}
```

💡 El acumulador `suma` empieza en 0 y va guardando el total. El patrón acumulador es uno de los más usados en programación.

15 Factorial de un número

PROBLEMA

Lee un número N y calcula su factorial ($N! = N \times (N-1) \times \dots \times 1$).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long long factorial = 1;
    cout << "Ingresa N: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        factorial *= i;
    }
    cout << n << "! = " << factorial << endl;
    return 0;
}
```

💡 Aquí el acumulador empieza en 1 porque vamos a multiplicar. Usamos `long long` porque el factorial crece muy rápido.

16 Serie de Fibonacci

PROBLEMA

Muestra los primeros N términos de la serie de Fibonacci: 0, 1, 1, 2, 3, 5, 8, ...

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Cuantos terminos? ";
    cin >> n;

    long long a = 0, b = 1;
    for (int i = 1; i <= n; i++) {
        cout << a << " ";
        long long siguiente = a + b;
        a = b;
        b = siguiente;
    }
    cout << endl;
    return 0;
}
```

💡 Cada término es la suma de los dos anteriores. Guardamos dos valores (`a` y `b`) y los vamos desplazando en cada vuelta.

17 ¿Es número primo?

PROBLEMA

Lee un número e indica si es primo (solo divisible entre 1 y él mismo).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Ingrese un numero: ";
    cin >> n;

    bool esPrimo = true;
    if (n < 2) esPrimo = false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            esPrimo = false;
            break;
        }
    }

    if (esPrimo) cout << n << " es primo" << endl;
    else cout << n << " no es primo" << endl;
    return 0;
}
```

💡 Solo necesitamos probar divisores hasta la raíz cuadrada (`i*i <= n`), lo que hace el programa mucho más rápido. El `break` corta el bucle apenas hallamos un divisor.

18 Invertir un número

PROBLEMA

Lee un número entero e imprímelo al revés (ej: 1234 → 4321).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n, invertido = 0;
    cout << "Ingrese un numero: ";
    cin >> n;

    while (n > 0) {
        int digito = n % 10; // ultimo digito
        invertido = invertido * 10 + digito;
        n = n / 10; // quitamos el ultimo digito
    }
    cout << "Invertido: " << invertido << endl;
    return 0;
}
```

💡 Con `% 10` obtenemos el último dígito y con `/ 10` lo eliminamos. El bucle `while` se repite hasta que el número llega a 0.

19 Contar los dígitos

PROBLEMA

Lee un número entero y cuenta cuántos dígitos tiene.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n, contador = 0;
    cout << "Ingrese un numero: ";
    cin >> n;
    if (n == 0) contador = 1;
    while (n > 0) {
        contador++;
        n /= 10;
    }
    cout << "Tiene " << contador << " digitos" << endl;
    return 0;
}
```

💡 Dividimos entre 10 repetidamente; cada división elimina un dígito. Tratamos el 0 como caso especial porque tiene un dígito.

SECCIÓN 4

Nivel 4 — Funciones

Organiza tu código en bloques reutilizables. Incluye recursividad.

20 Función potencia

PROBLEMA

Crea una función `potencia(base, exp)` que calcule base elevado a exp, y úsala desde main.

SOLUCIÓN

```
#include <iostream>
using namespace std;

long long potencia(int base, int exp) {
    long long resultado = 1;
    for (int i = 1; i <= exp; i++) {
        resultado *= base;
    }
    return resultado;
}

int main() {
    int b, e;
    cout << "Ingrese base y exponente: ";
    cin >> b >> e;
    cout << b << "^" << e << " = " << potencia(b, e) << endl;
    return 0;
}
```

💡 Una función recibe parámetros y devuelve un valor con `return`. Definirla fuera de `main` permite reutilizarla cuantas veces queramos.

21 Máximo común divisor (MCD)

PROBLEMA

Escribe una función que calcule el MCD de dos números usando el algoritmo de Euclides.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int mcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    int x, y;
    cout << "Ingrese dos numeros: ";
    cin >> x >> y;
    cout << "MCD = " << mcd(x, y) << endl;
    return 0;
}
```

💡 El algoritmo de Euclides reemplaza repetidamente el par (a, b) por (b, a%b) hasta que b sea 0. El MCD es el último valor de `a`.

22 Verificar número palíndromo

PROBLEMA

Crea una función booleana que indique si un número se lee igual de izquierda a derecha que al revés (ej: 12321).

SOLUCIÓN

```
#include <iostream>
using namespace std;

bool esPalindromo(int n) {
    int original = n, invertido = 0;
    while (n > 0) {
        invertido = invertido * 10 + n % 10;
        n /= 10;
    }
    return original == invertido;
}

int main() {
    int n;
    cout << "Ingrese un numero: ";
    cin >> n;
    if (esPalindromo(n)) cout << "Es palindromo" << endl;
    else cout << "No es palindromo" << endl;
    return 0;
}
```

💡 Reutilizamos la técnica de invertir un número. Una función `bool` devuelve verdadero o falso, ideal para verificaciones.

23 Factorial recursivo

PROBLEMA

Calcula el factorial usando una función que se llama a sí misma (recursividad).

SOLUCIÓN

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    if (n <= 1) return 1; // caso base
    return n * factorial(n - 1); // caso recursivo
}

int main() {
    int n;
    cout << "Ingrese N: ";
    cin >> n;
    cout << n << " != " << factorial(n) << endl;
    return 0;
}
```

💡 Toda función recursiva necesita un **caso base** que detenga las llamadas. Aquí, `n!` se define como `n × (n-1)!` hasta llegar a 1.

SECCIÓN 5

Nivel 5 — Arreglos (vectores)

Trabaja con listas de datos: recorrer, buscar y ordenar.

24 Llenar e imprimir un arreglo

PROBLEMA

Lee N números, guárdalos en un arreglo e imprímelos.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Cuantos numeros? ";
    cin >> n;
    int arr[100];    // capacidad maxima

    for (int i = 0; i < n; i++) {
        cout << "Elemento " << i << ": ";
        cin >> arr[i];
    }

    cout << "Arreglo: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

💡 Un arreglo guarda muchos valores bajo un mismo nombre. Se accede por índice empezando en **0**: `arr[0]` es el primero.

25 Mayor y menor de un arreglo

PROBLEMA

Lee N números y muestra el mayor y el menor.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Cuantos numeros? ";
    cin >> n;
    int arr[100];
    for (int i = 0; i < n; i++) cin >> arr[i];

    int mayor = arr[0], menor = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > mayor) mayor = arr[i];
        if (arr[i] < menor) menor = arr[i];
    }
    cout << "Mayor: " << mayor << ", Menor: " << menor << endl;
    return 0;
}
```

💡 Inicializamos mayor y menor con el primer elemento y recorremos el resto comparando. Nunca inicialices el mayor en 0: fallaría con números negativos.

26 Suma y promedio de un arreglo

PROBLEMA

Lee N números y calcula la suma total y el promedio.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Cuantos numeros? ";
    cin >> n;
    double arr[100], suma = 0;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        suma += arr[i];
    }
    cout << "Suma: " << suma << endl;
    cout << "Promedio: " << suma / n << endl;
    return 0;
}
```

💡 Aprovechamos el mismo bucle de lectura para ir acumulando la suma, evitando recorrer el arreglo dos veces.

27 Búsqueda lineal

PROBLEMA

Lee un arreglo y un valor a buscar; indica en qué posición se encuentra (o que no existe).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n, buscado;
    cout << "Cuantos numeros? ";
    cin >> n;
    int arr[100];
    for (int i = 0; i < n; i++) cin >> arr[i];

    cout << "Que numero buscas? ";
    cin >> buscado;

    int posicion = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] == buscado) {
            posicion = i;
            break;
        }
    }

    if (posicion != -1) cout << "Encontrado en la posicion " << posicion << endl;
    else cout << "No se encuentra en el arreglo" << endl;
    return 0;
}
```

💡 Usamos `-1` como marca de "no encontrado". Si hallamos el valor, guardamos su índice y cortamos con `break`.

28 Ordenar con método burbuja

PROBLEMA

Ordena un arreglo de menor a mayor usando el algoritmo de la burbuja.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Cuantos numeros? ";
    cin >> n;
    int arr[100];
    for (int i = 0; i < n; i++) cin >> arr[i];

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    cout << "Ordenado: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

💡 El método burbuja compara elementos vecinos y los intercambia si están en desorden. Tras cada pasada, el mayor "sube" a su posición final.

29 Invertir un arreglo

PROBLEMA

Lee un arreglo e imprímelo en orden inverso intercambiando elementos.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Cuantos numeros? ";
    cin >> n;
    int arr[100];
    for (int i = 0; i < n; i++) cin >> arr[i];

    for (int i = 0; i < n / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }

    cout << "Invertido: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

💡 Intercambiamos el primero con el último, el segundo con el penúltimo, etc. Solo recorreremos hasta la mitad ($n/2$).

SECCIÓN 6

Nivel 6 — Matrices (avanzado)

Arreglos de dos dimensiones: el reto final del libro.

30 Llenar e imprimir una matriz

PROBLEMA

Lee una matriz de F filas por C columnas e imprímela con formato de tabla.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int f, c;
    cout << "Filas y columnas: ";
    cin >> f >> c;
    int m[50][50];

    // Lectura
    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++) {
            cout << "m[" << i << "][" << j << "] = ";
            cin >> m[i][j];
        }

    // Impresion
    cout << "\nMatriz:" << endl;
    for (int i = 0; i < f; i++) {
        for (int j = 0; j < c; j++)
            cout << m[i][j] << "\t";
        cout << endl;
    }
    return 0;
}
```

🔗 Una matriz necesita **dos índices**: **i** para la fila y **j** para la columna. Por eso usamos dos bucles **for** anidados.

31 Suma de dos matrices

PROBLEMA

Lee dos matrices del mismo tamaño y muestra la matriz resultante de sumarlas.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int f, c;
    cout << "Filas y columnas: ";
    cin >> f >> c;
    int a[50][50], b[50][50], suma[50][50];

    cout << "Matriz A:" << endl;
    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++) cin >> a[i][j];

    cout << "Matriz B:" << endl;
    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++) cin >> b[i][j];

    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++)
            suma[i][j] = a[i][j] + b[i][j];

    cout << "Suma:" << endl;
    for (int i = 0; i < f; i++) {
        for (int j = 0; j < c; j++) cout << suma[i][j] << "\t";
        cout << endl;
    }
    return 0;
}
```

🔗 Para sumar matrices se suma posición por posición: `suma[i][j] = a[i][j] + b[i][j]`. Ambas deben tener el mismo tamaño.

32 Multiplicación de matrices

PROBLEMA

Multiplica una matriz A ($f1 \times c1$) por una matriz B ($c1 \times c2$). Recuerda: las columnas de A deben coincidir con las filas de B.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int f1, c1, c2;
    cout << "Filas y columnas de A: ";
    cin >> f1 >> c1;
    cout << "Columnas de B: ";
    cin >> c2;          // las filas de B = c1

    int a[50][50], b[50][50], r[50][50];

    cout << "Matriz A:" << endl;
    for (int i = 0; i < f1; i++)
        for (int j = 0; j < c1; j++) cin >> a[i][j];

    cout << "Matriz B:" << endl;
    for (int i = 0; i < c1; i++)
        for (int j = 0; j < c2; j++) cin >> b[i][j];

    for (int i = 0; i < f1; i++)
        for (int j = 0; j < c2; j++) {
            r[i][j] = 0;
            for (int k = 0; k < c1; k++)
                r[i][j] += a[i][k] * b[k][j];
        }

    cout << "Producto:" << endl;
    for (int i = 0; i < f1; i++) {
        for (int j = 0; j < c2; j++) cout << r[i][j] << "\t";
        cout << endl;
    }
    return 0;
}
```

💡 Cada elemento del resultado es la suma de productos de una fila de A por una columna de B. Por eso se necesitan **tres** bucles anidados (i, j, k).

33 Transpuesta de una matriz

PROBLEMA

Lee una matriz y muestra su transpuesta (las filas se convierten en columnas).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int f, c;
    cout << "Filas y columnas: ";
    cin >> f >> c;
    int m[50][50], t[50][50];

    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++) cin >> m[i][j];

    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++)
            t[j][i] = m[i][j];    // se invierten los indices

    cout << "Transpuesta:" << endl;
    for (int i = 0; i < c; i++) {
        for (int j = 0; j < f; j++) cout << t[i][j] << "\t";
        cout << endl;
    }
    return 0;
}
```

💡 Transponer es intercambiar índices: el elemento `m[i][j]` pasa a la posición `t[j][i]`. La transpuesta tiene C filas y F columnas.

34 Suma de la diagonal principal

PROBLEMA

Lee una matriz cuadrada ($N \times N$) y suma los elementos de su diagonal principal.

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Tamano de la matriz cuadrada: ";
    cin >> n;
    int m[50][50], suma = 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) cin >> m[i][j];

    for (int i = 0; i < n; i++)
        suma += m[i][i]; // diagonal: fila == columna

    cout << "Suma de la diagonal: " << suma << endl;
    return 0;
}
```

💡 En la diagonal principal el índice de fila es igual al de columna, por eso basta con `m[i][i]` y un solo bucle.

35 Mayor elemento de la matriz

PROBLEMA

Lee una matriz y muestra el valor mayor junto con su posición (fila y columna).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int f, c;
    cout << "Filas y columnas: ";
    cin >> f >> c;
    int m[50][50];
    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++) cin >> m[i][j];

    int mayor = m[0][0], fila = 0, col = 0;
    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++)
            if (m[i][j] > mayor) {
                mayor = m[i][j];
                fila = i;
                col = j;
            }

    cout << "Mayor: " << mayor;
    cout << " en la posicion [" << fila << "][" << col << "]" << endl;
    return 0;
}
```

💡 Combinamos la búsqueda del mayor (Nivel 5) con matrices. Guardamos no solo el valor sino también dónde se encontró.

36 ¿Es matriz identidad?

PROBLEMA

Lee una matriz cuadrada y verifica si es la matriz identidad (unos en la diagonal y ceros en el resto).

SOLUCIÓN

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Tamano (N): ";
    cin >> n;
    int m[50][50];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) cin >> m[i][j];

    bool esIdentidad = true;
    for (int i = 0; i < n && esIdentidad; i++)
        for (int j = 0; j < n; j++) {
            if (i == j && m[i][j] != 1) esIdentidad = false;
            if (i != j && m[i][j] != 0) esIdentidad = false;
        }

    if (esIdentidad) cout << "Es matriz identidad" << endl;
    else cout << "No es matriz identidad" << endl;
    return 0;
}
```

💡 Recorremos toda la matriz: en la diagonal (`i == j`) debe haber 1 y fuera de ella 0. Una sola violación basta para descartarla.

¡Felicidades! 🎉

Si llegaste hasta aquí resolviendo los ejercicios, ya tienes una base sólida en C++.

El siguiente paso es construir tus propios proyectos. Sigue practicando.

ÚNETE A LA COMUNIDAD



Suscríbete en YouTube — Miguel Borja Academy

youtube.com/@MiguelBorjaAcademy



Sígueme en GitHub

github.com/migueldevbp



Dale una estrella al repositorio de este libro

github.com/migueldevbp/c-workbook

Comparte este material con quien lo necesite. ¡Programar se aprende programando!